Ninth-Degree Polynomial Theory

Intro And Scope

This document introduces the ninth-degree polynomial as the first complete reference case for the project. Its role is twofold:

- to define the polynomial in forms that are useful for derivation and implementation;
- to study the derivative chain from position up to the sixth derivative, so the later trajectory-planning documents can refer to a shared theoretical base.

The treatment starts in 1D, because that is the base case adopted by the project. The 3D case is then obtained by applying the same scalar law independently to the three Cartesian components.

This document focuses on Requirement 2 of `01-requirements.md`:

- canonical form and Horner form in the time domain;
- derivatives up to the sixth derivative;
- discussion of the usefulness of using more or fewer derivatives in trajectory planning.

Symbols Used

The notation follows `SymbolsAndNomenclature.md`. The main symbols used here are:

- t: absolute time;
- t_0: initial time;
- t_f: final time;
- $T = t_f - t_0$: motion duration;
- $x(t)$: scalar position law in 1D;
- $a_0, a_1, \dots, a_9$: polynomial coefficients;
- $v(t), a(t), j(t)$: velocity, acceleration, and jerk;
- $x^{(4)}(t), x^{(5)}(t), x^{(6)}(t)$: snap, crackle, and pop.

When normalized time is briefly mentioned, the symbol τ is used with $\tau \in [0, 1]$.

Why Degree Nine Is A Natural First Case

A ninth-degree polynomial has ten coefficients. This makes it especially useful as a first full study case because a trajectory segment can be determined by ten scalar conditions if the formulation requires them.

In the standard symmetric endpoint formulation, those ten conditions are naturally obtained by prescribing:

$$x_0,\ x_f,\ v_0,\ v_f,\ a_0,\ a_f,\ j_0,\ j_f,\ s_0,\ s_f$$

where s denotes snap.

This does not mean that every problem must always be posed this way. It means that degree nine is the first degree in this project that naturally accommodates endpoint information up to snap at both ends of a segment.

Canonical Form In The Time Domain

The canonical form is the direct polynomial expansion in powers of time:

$$x(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3 + a_4 t^4 + a_5 t^5 + a_6 t^6 + a_7 t^7 + a_8 t^8 + a_9 t^9$$

The same expression can be written compactly as:

$$x(t) = \sum_{i=0}^{9} a_i t^i$$

This form is the most transparent one for theoretical work because every coefficient is visibly attached to its corresponding power of time.

Reading The Canonical Form

The canonical form is useful to read the structure of the motion law step by step:

1. the constant term a_0 is the value present even at $t = 0$;
2. the linear term $a_1 t$ contributes directly to initial slope;
3. higher powers progressively shape curvature and higher-order changes;
4. the highest-degree term $a_9 t^9$ gives the polynomial its ninth-degree character.

This reading is qualitative, but it is educationally useful because it connects algebraic structure to motion behavior.

Horner Form In The Time Domain

The same polynomial can be rewritten in nested form:

$$x(t) = a_0 + t\Big(a_1 + t(a_2 + t(a_3 + t(a_4 + t(a_5 + t(a_6 + t(a_7 + t(a_8 + ta_9)))))))\Big)$$

Equivalent right-nested notation is:

$$x(t) = (((((((((a_9 t + a_8)t + a_7)t + a_6)t + a_5)t + a_4)t + a_3)t + a_2)t + a_1)t + a_0)$$

These two expressions are algebraically identical. The difference is only in how the polynomial is organized for reading and evaluation.

How Horner Form Is Built

Starting from the canonical form:

$$x(t) = a_0 + a_1 t + a_2 t^2 + \cdots + a_9 t^9$$

factor t progressively:

$$x(t) = a_0 + t\left(a_1 + a_2 t + a_3 t^2 + \cdots + a_9 t^8\right)$$

then factor t again inside the parentheses:

$$x(t) = a_0 + t\left(a_1 + t\left(a_2 + a_3 t + a_4 t^2 + \cdots + a_9 t^7\right)\right)$$

and continue until the fully nested structure is obtained.

This is not a new polynomial. It is the same polynomial rewritten so that each step uses one multiplication by t and one addition.

Canonical Form Vs Horner Form

The two forms serve different purposes.

Canonical Form: Advantages

- It shows the polynomial degree immediately.
- It makes symbolic differentiation straightforward.
- It is convenient when setting up coefficient-determination equations.
- It is easier to inspect manually during theoretical study.
- It exposes clearly which coefficient multiplies which power of time.

Canonical Form: Limitations

- It is less efficient for repeated numerical evaluation.
- A direct implementation may compute many powers of t explicitly.
- Large powers of time can make numerical scaling less convenient.
- It is less natural for recursive evaluation.

Horner Form: Advantages

- It is computationally efficient for numerical evaluation.
- It avoids explicit computation of powers such as t^7, t^8, and t^9.
- It reduces the number of multiplications needed.
- It is often better suited for implementation in software.
- It is easy to evaluate repeatedly for many time samples.

Horner Form: Limitations

- It hides the power-by-power structure of the polynomial.
- It is less intuitive when deriving equations by hand.
- It is less transparent for teaching coefficient meaning.
- It is not the most convenient form for symbolic derivations.

Practical Interpretation

For this project, the best attitude is not to treat the two forms as competitors, but as complementary tools:

- canonical form is the preferred form for theory, derivation, and boundary-condition reasoning;
- Horner form is the preferred form for efficient numerical evaluation in software.

This is an important educational point: different mathematical forms of the same object become useful at different stages of the project.

General Derivative Rule

If

$$x(t) = \sum_{i=0}^{9} a_i t^i$$

then the k-th derivative is:

$$x^{(k)}(t) = \sum_{i=k}^{9} \frac{i!}{(i-k)!} a_i t^{i-k}$$

This compact formula is useful because it shows the pattern behind all derivative expressions:

- every differentiation lowers the power of t by one;
- every differentiation multiplies the coefficient by the current exponent;
- terms of degree lower than k disappear in the k-th derivative.

Derivatives Of The Ninth-Degree Polynomial

Starting from

$$x(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3 + a_4 t^4 + a_5 t^5 + a_6 t^6 + a_7 t^7 + a_8 t^8 + a_9 t^9$$

the derivatives up to the sixth are the following.

First Derivative: Velocity

$$v(t) = \dot{x}(t) = a_1 + 2a_2 t + 3a_3 t^2 + 4a_4 t^3 + 5a_5 t^4 + 6a_6 t^5 + 7a_7 t^6 + 8a_8 t^7 + 9a_9 t^8$$

Velocity is the first quantity that describes how position changes in time. It is also the most immediate kinematic limit in many planning problems.

Second Derivative: Acceleration

$$a(t) = \ddot{x}(t) = 2a_2 + 6a_3 t + 12a_4 t^2 + 20a_5 t^3 + 30a_6 t^4 + 42a_7 t^5 + 56a_8 t^6 + 72a_9 t^7$$

Acceleration measures how velocity changes. It is central whenever smooth dynamic evolution is more important than position alone.

Third Derivative: Jerk

$$j(t) = x^{(3)}(t) = 6a_3 + 24a_4 t + 60a_5 t^2 + 120a_6 t^3 + 210a_7 t^4 + 336a_8 t^5 + 504a_9 t^6$$

Jerk is often introduced when acceleration continuity is not sufficient and sharper changes need to be controlled.

Fourth Derivative: Snap

$$x^{(4)}(t) = 24a_4 + 120a_5 t + 360a_6 t^2 + 840a_7 t^3 + 1680a_8 t^4 + 3024a_9 t^5$$

Snap is especially relevant in a ninth-degree formulation because endpoint snap values can be included naturally among the ten scalar conditions.

Fifth Derivative: Crackle

$$x^{(5)}(t) = 120a_5 + 720a_6 t + 2520a_7 t^2 + 6720a_8 t^3 + 15120a_9 t^4$$

Crackle is less commonly used as a physical planning quantity, but it remains analytically useful because it is the derivative of snap.

Sixth Derivative: Pop

$$x^{(6)}(t) = 720a_6 + 5040a_7 t + 20160a_8 t^2 + 60480a_9 t^3$$

Pop is the derivative of crackle. It is rarely a primary planning constraint, but it is part of the natural derivative chain of a ninth-degree polynomial and can help understand higher-order variation.

What Changes In 3D

The 3D case does not introduce a new scalar theory. It applies the same ninth-degree law to each Cartesian component:

$$\mathbf{p}(t) = \begin{bmatrix} x(t) \\ y(t) \\ z(t) \end{bmatrix}$$

with

$$x(t) = \sum_{i=0}^{9} a_i^{(x)} t^i, \qquad y(t) = \sum_{i=0}^{9} a_i^{(y)} t^i, \qquad z(t) = \sum_{i=0}^{9} a_i^{(z)} t^i$$

The same derivative chain applies component-wise:

$$\dot{\mathbf{p}}(t), \qquad \ddot{\mathbf{p}}(t), \qquad \mathbf{p}^{(3)}(t), \qquad \mathbf{p}^{(4)}(t), \qquad \mathbf{p}^{(5)}(t), \qquad \mathbf{p}^{(6)}(t)$$

So, from the viewpoint of this document:

- the 1D theory is the real mathematical core;
- the 3D extension is obtained by repeating that theory on three components;
- extra difficulty in 3D comes later from coupling choices, shared duration, and constraint interpretation, not from a different polynomial basis.

Why Fewer Derivatives May Be Enough

In many trajectory-planning problems, using fewer derivatives is entirely reasonable.

Position And Velocity Only

If the problem only requires reaching a target position with prescribed endpoint velocities, a lower-order description may be sufficient. This is useful when:

- the motion is simple;
- dynamic smoothness beyond velocity is not critical;
- the polynomial degree can be kept lower.

The advantage is simplicity. The limitation is that acceleration and higher-order behavior are left less controlled.

Up To Acceleration

Using position, velocity, and acceleration is common when smoother motion is needed. This usually gives a better compromise between control and complexity.

Acceleration-level reasoning is already very meaningful because:

- many mechanical effects are more closely related to acceleration than to position alone;
- acceleration continuity usually improves motion quality substantially;
- it avoids some abrupt changes that remain hidden if only velocity is considered.

Up To Jerk

Including jerk is often valuable when the transition smoothness itself becomes a design concern. Even without discussing actuators in detail, jerk remains educationally important because it captures how aggressively acceleration changes.

For this project, jerk is especially useful because it forms the bridge between basic trajectory geometry and genuinely smooth motion planning.

Why More Derivatives Can Be Useful

Studying derivatives beyond jerk is not always necessary, but it can still be useful for at least three reasons.

First Reason: Endpoint Specification Richness

Degree nine allows endpoint conditions up to snap at both ends. If snap is part of the formulation, then studying snap explicitly is not optional; it belongs to the definition of the segment itself.

Second Reason: Higher-Order Smoothness

Even if crackle and pop are not imposed as boundary conditions, they still describe how higher-order quantities evolve. This is useful when the goal is not only to obtain a valid trajectory, but to understand how smooth it really is.

Third Reason: Analytical Support For Later Constraints

Higher derivatives appear naturally when later documents analyze extrema:

- extrema of velocity are related to acceleration;
- extrema of acceleration are related to jerk;
- extrema of jerk are related to snap;
- extrema of snap are related to crackle.

So even if crackle and pop are not used as physical constraints, they remain mathematically useful.

Are Crackle And Pop Physically Necessary

Usually, no.

For most practical planning formulations, crackle and pop are not primary quantities to prescribe directly. In this project they are mainly useful because:

- they complete the derivative hierarchy of the ninth-degree polynomial;
- they help interpret the behavior of snap and higher-order smoothness;
- they support later analytical reasoning about maxima, minima, and continuity.

In other words:

- position, velocity, acceleration, and jerk are often directly meaningful;
- snap may become directly meaningful in high-smoothness formulations;
- crackle and pop are often more useful as analytical companions than as primary design targets.

The Special Role Of Snap In Degree Nine

Among the higher derivatives, snap has a special status in this project.

The reason is structural: a ninth-degree polynomial has ten coefficients, and the symmetric endpoint data set

$$(x_0, v_0, a_0, j_0, s_0), \qquad (x_f, v_f, a_f, j_f, s_f)$$

provides exactly ten scalar conditions.

This makes snap the highest derivative that fits naturally into the standard two-endpoint determination of a ninth-degree segment.

That is why this document studies derivatives beyond jerk, but still treats snap as the highest derivative with a particularly strong structural role.

A Practical Evaluation Procedure

When the polynomial must be evaluated numerically at many time instants, the following workflow is useful:

1. derive the equations in canonical form;
2. determine the coefficients from the chosen boundary conditions;
3. convert the polynomial to Horner form for repeated evaluation;
4. sample the motion over time to obtain position and derivative profiles.

This workflow matches the educational goal of the project:

- theory remains readable in canonical form;
- implementation remains efficient in Horner form.

Worked Symbolic Considerations

Example 1: Full Ninth-Degree Segment In 1D

Consider the symbolic segment

$$x(t) = \sum_{i=0}^{9} a_i t^i$$

with endpoint data prescribed up to snap at both ends.

The key observation is not yet the solution for the coefficients, but the structural match:

- unknowns: $a_0, a_1, \dots, a_9$;
- scalar equations: initial and final values of position, velocity, acceleration, jerk, and snap.

This is why the ninth-degree polynomial is a natural theoretical anchor for the rest of the project.

Example 2: Why Higher Derivatives Matter For Extrema

Suppose a later document needs the maximum snap value. Then the stationary points of snap are found from:

$$\frac{d}{dt}\left(x^{(4)}(t)\right) = x^{(5)}(t) = 0$$

So crackle appears naturally even if it was never chosen as a planning constraint.

Example 3: 3D Interpretation

If

$$\mathbf{p}(t) = \begin{bmatrix} x(t) \\ y(t) \\ z(t) \end{bmatrix}$$

then a ninth-degree 3D segment is simply three synchronized ninth-degree scalar segments. The scalar theory does not change; only the bookkeeping grows.

Open Questions

- In later implementation, should derivative profiles be evaluated directly from canonical formulas or from Horner-style recursive schemes?
- When extending from 1D to 3D, should limits be interpreted component-wise or on vector magnitude?
- In the final application, which derivative levels should be visible by default in the charts, and which should remain optional?

Related Documents

- SymbolsAndNomenclature.md
- BoundaryConditions
- PolynomialCoefficientDetermination
- TrajectoryConstraints